
Recomendaciones generales

Esta guía contiene un trabajo práctico de Métodos Analíticos y Programación. El mismo debe ser resuelto íntegramente en **Python**. La resolución no es simple y se espera que resolverlo tome cierto tiempo por fuera de las clases.

Recuerden que pueden consultarlos sin ningún tipo de problema y que este trabajo y los siguientes son parte de la evaluación que tendrán de la materia. Se sugiere que **empiecen a resolver el TP con tiempo** de modo de tener tiempo para hacer consultas y no estar apurados a último momento.

El Trabajo Práctico debe resolverse de a grupos de **2 o 3 alumnos, no más y no menos**. Las consultas podrán realizarse por mail y en los momentos asignados en clase. Tener en cuenta que **si la consulta es realizada a último momento puede no ser factible responderla**. **No está permitida la interacción entre grupos**.

MAYP YouTubeBot: Creando bots que simulan comentarios de YouTube

En este trabajo práctico vamos a tratar de combinar datos reales, modelos matemáticos, procesamiento del lenguaje y programación para generar un *bot* (i.e., un programa) que pueda *aprender* e imitar la forma de construir frases (i.e., comentarios de YouTube) de un grupo determinado. Nuestro objetivo consiste en formular un prototipo que podría ser utilizado por distinto tipo de organizaciones para generar contenidos en base a un conjunto de datos no estructurados.

La Figura 1 muestra algunos ejemplos de comentarios de usuarios de distintos videos, en particular, un video de política y uno de un partido de fútbol. Youtube es una plataforma cuyos usuarios acceden con distintas intenciones de uso, y por lo tanto muestran comportamientos diferentes. Esto también se ve reflejado en el tipo, la estructura y otras características del contenido generado, que no es trivial de capturar y entender por un programa. Los comentarios pueden tener opiniones personales, ironías, intercambios con preguntas y respuestas encadenadas, etc. En este sentido, el *bot* que generemos intentará aprender en base a un video o grupo de videos las reglas del público de construcción de textos en general, para luego generar de manera automatizada e independiente nuevos contenidos. Es importante destacar que los métodos aplicados en este trabajo han sido considerado *estado del arte* en la temática hasta hace algunos años, previo a la irrupción de métodos más sofisticados. A su vez, estas técnicas pueden ser aplicadas en otros contextos distintos, como el *predictivo* de los teclados de los teléfonos móviles o auto correctores de mensajes de texto. Adicionalmente, el entorno y metodología de trabajo aplicados son consistentes con los usados en la práctica, salvo algunas simplificaciones.

El modelo

Para construir el *bot* generador de comentarios de YouTube vamos a implementar un *Modelo de Lenguaje* basado en *n-grams* y *Modelos de Markov* usando grafos y simulación. Resumi-

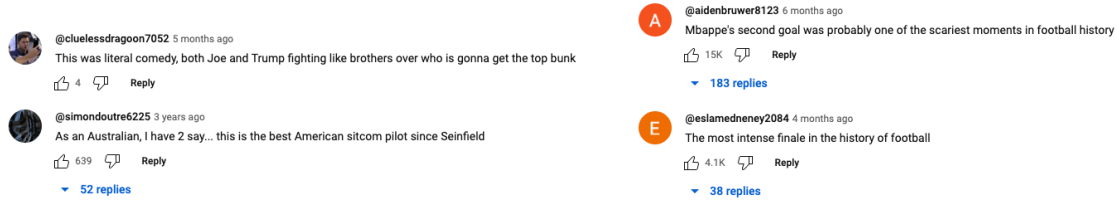


Figura 1: Ejemplos de comentarios de distintos tipos de videos.

damente, el modelo de n -grams asigna probabilidades a oraciones y secuencias de palabras según sus ocurrencias en datos usados como entrada para que el modelo aprenda. A continuación incluimos una breve explicación del modelo, y luego las indicaciones respecto a la implementación del mismo. Consideremos a modo de ejemplo el siguiente extracto del script del capítulo *The Springfield Files*:

He walks down eerie roads. He hears scary music.

Un n -gram es una secuencia de n palabras consecutivas. El caso más simple es el llamado *bigrama*, que es una secuencia de 2 palabras consecutivas. En base al ejemplo, podemos encontrar los bigramas *He walks*, *walks down*, *down eerie*, entre otros. Notar que el orden es relevante en la secuencia, y por lo tanto *walks He* no es un bigrama presente en el ejemplo. La idea de bigramas puede generalizarse a secuencias de tamaño n , como por ejemplo, para $n = 3$, *He walks down* y *walks down eerie*.

El objetivo del modelo de lenguaje es tratar de estimar la probabilidad de ocurrencia de la última palabra de un n -gram dadas las anteriores. En particular para los bigramas y los ejemplos vistos, supongamos que queremos estimar la probabilidad de que *walks* y *hears* sigan a *He*, entre otros. Formalmente, nos interesa estimar $P(walks|He)$ y $P(hears|He)$. Intuitivamente, esto podemos hacerlo simplemente tomando la proporción entre casos favorables y casos totales en el total de inputs que tengamos, es decir, contar

$$P(walks|He) = \frac{\text{cantidad de ocurrencias } He \text{ walks}}{\text{cantidad de ocurrencias } He}$$

Para generar la frase con el bot no solo nos importan las probabilidades, sino también la secuencia de aparición de las palabras. Para ello, incorporamos al modelo del lenguaje un grafo dirigido (digrafo), al que incorporaremos las probabilidades como pesos, a fin de capturar también las secuencias factibles que pueden generarse en base a los textos utilizados para aprender. Para el caso de bigramas, definimos un digrafo $D = (V, A)$ con las siguientes características:

- V es el conjunto de vértices, y tendrá un vértice por cada palabra¹ en el texto. Los signos de puntuación y algunos caracteres especiales (como usuarios o hashtags) también serán considerados palabras. Los vértices serán directamente **strings**. A modo de ejemplo, *He*, *walks*, *hears* serán vértices en V .

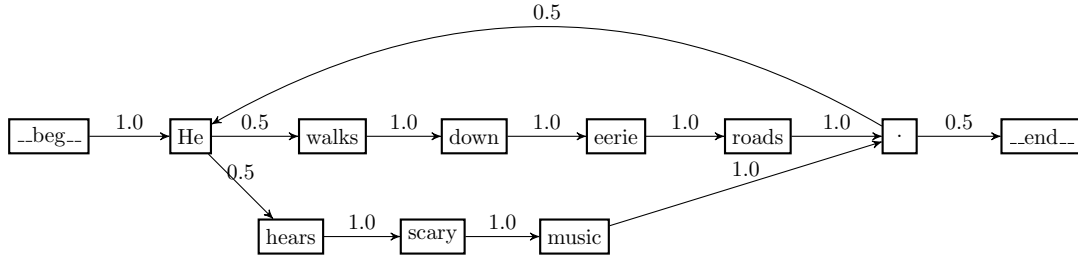
¹El concepto de palabra es genérico, y en la práctica una librería obtendrá las *palabras* en base al texto.

- A es el conjunto de arcos, tendremos un arco $(i, j) \in A$ si $i j$ es un bigrama presente en los textos. A modo de ejemplo, de los bigramas *He walks* y *He hears* obtendremos los arcos $(He, walks)$ y $(He, hears)$. Dado que consideramos un digrafo y el bigrama *walks He* no pertenece al texto, entonces el eje $(walks, He)$ no existirá en A .
- Para cada arco $(i, j) \in A$ definiremos un peso w_{ij} con la probabilidad condicional $P(i|j)$ mencionada anteriormente. A modo de ejemplo, $w_{He, walks} = w_{He, hears} = 0,5$ y $w_{down, eerie} = 1,0$.

Un aspecto que hasta ahora no mencionamos es identificar el inicio y fin del texto a generar. Esto se suele hacer en base a un preprocesamiento simple del texto agregando indicadores de inicio y fin a cada unidad de texto del conjunto de input (en nuestro caso, en cada comentario). Para que no se genere conflictos con el texto, definimos `__beg__` como indicador de inicio y `__end__` como indicador de fin. Continuando con el ejemplo, para cada unidad de texto (nosotros tenemos una sola) el texto se transforma de la siguiente manera:

`__beg__ He walks down eerie roads. He hears scary music. __end__`

Los indicadores de inicio y fin los consideraremos como si fuesen una palabra más, y aplica exactamente el mismo análisis que antes. Considerando el ejemplo planteado en esta sección, el digrafo final es el siguiente:



Una vez que construimos el modelo, un texto que sigue las reglas del lenguaje modelado (en nuestro caso, el público de un cierto video) generando un *camino aleatorio* que empiece en `__beg__` y termine en `__end__`. Para ello, iniciamos en el vértice `__beg__` y en cada paso elegimos la siguiente palabra de forma aleatoria entre los *sucesores* del vértice actual. A modo de ejemplo, supongamos que hasta ahora generamos el texto `__beg__ He`, y estamos en el vértice `He`. Luego, la siguiente palabra será `walks` o `hears`, ambos con probabilidad 0.5. Elegimos de manera aleatoria, y supongamos que resulta elegida `walks`. Entonces, el nuevo texto parcial es `__beg__ He walks` (notar que agregamos un espacio entre las palabras, y ahora estamos en el vértice `walks` donde tenemos que repetir el mismo procedimiento. La simulación termina una vez que llegamos al vértice `__end__`, que por construcción no tiene ejes salientes.

El modelo descrito anteriormente no considera mucha información del contexto, ya que la siguiente palabra (y su correspondiente probabilidad) depende únicamente de la palabra previa. Una forma de incorporar cierto contexto al modelo es hacer que la probabilidad de aparición de la última palabra dependa de una secuencia de palabras consecutivas. En particular, en el trabajo vamos a generalizar esta idea y a considerar que la probabilidad de aparición de una palabra dependa de bigramas, cuyo impacto en la implementación es mínimo.

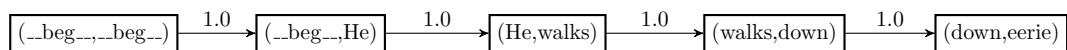
Consideramos los ejemplos de trigramas presentados anteriormente: *He walks down* y *walks down eerie*. El contexto puede expandirse considerando $P(\text{down}|\text{He walks})$ y $P(\text{eerie}|\text{walks down})$, es decir, ahora la aparición de una palabra (y su probabilidad) depende de la secuencia previa de dos palabras. La forma de estimar estas probabilidades condicionales es análoga a la anterior.

El armado del digrafo también es similar al caso previo. Si bien existen distintas alternativas, una forma genérica consiste en:

- El texto inicialmente se transforma duplicando los indicadores de principio y de fin (esto simplifica pasos posteriores y evita considerar casos diferentes)

`__beg__ __beg__ He walks down eerie roads. He hears scary music. __end__ __end__`

- Los vértices del conjunto V de D pasarán a ser tuplas de palabras (consecutivas) en lugar de una única palabra. Esto puede implementarse usando tuplas específicamente, o también otra alternativa es que sea un string con algún delimitador entre palabras. En base al ejemplo, los vértices del digrafo serán $(_beg_,_beg_)$, $(_beg_,\text{He})$, (He,walks) , $(\text{walks},\text{down})$, $(\text{down},\text{eerie})$.
- Los arcos del conjunto A de D conectarán vértices (pares de palabras) que compartan una de ellas, de cierta forma particular, a fin de capturar la siguiente palabra dado el contexto de las dos palabras anteriores. Considerando el texto y los nodos definidos en los puntos anteriores, el grafo a generar es el siguiente (mostrado de forma parcial):



Más formalmente, si el texto contiene la secuencia de palabras (o símbolos) $p_1 p_2 p_3$, se crearán los nodos (p_1, p_2) y (p_2, p_3) y tendremos el eje $((p_1, p_2), (p_2, p_3))$ en el grafo a fin de capturar del contexto que p_3 aparece luego de $p_1 p_2$. Notar que dado un arco $(i, j) \in A$ el manejo de los pesos w_{ij} se calcula de manera análoga al modelo anterior.

- Finalmente, un posible texto basado en nuestro modelo consiste en un camino (no necesariamente simple) del digrafo que comienza en $(_beg_,_beg_)$ y termina en $(_end_,_end_)$.

El trabajo

Este trabajo práctico va a estar dividido en cuatro etapas, simulando lo que podría ser un flujo posible al momento de abordar esta tarea en la práctica. Las etapas estimadas se muestran a continuación. Se remarca que si bien las mismas son presentadas de forma secuencial, en general el proceso suele incluir algún tipo de iteración hasta refinar el modelo y que los resultados sean los buscados.



A continuación explicamos los objetivos y lineamientos a seguir en cada una de ellas.

1. **Pregunta e hipótesis de análisis (1 punto).** Originalidad y relevancia del caso a estudiar. Justificar su elección, ya que no todos los videos generan contenidos replicables, en mayor o menor medida, con el método propuesto. También se considerará positivamente aquellos que propongan alternativas originales (como por ejemplo combinar videos de contextos diferentes).
2. **Bajada y procesamiento de datos (2 puntos).** Se centra en el proceso de obtención de los datos crudos de los videos elegidos y un primer procesamiento de extracción de información.

- *Obtención de datos.* Vamos a usar la librería `googleapiclient`², que nos permite acceder (entre otras cosas) a comentarios de un video de YouTube. Esta librería no viene instalada por defecto y se necesita una clave especial para utilizarla. El proceso de instalación y la configuración de la clave será mostrado durante la clase práctica.

No es obligatorio instalar la librería, ya que puede usarse como dataset algunos de los propuestos por los docentes para resolver el TP. De todas formas, el archivo `downloader.py` provee una implementación que permite bajar información de un video de YouTube simplemente modificando algunas variables de configuración en el `main`. Los resultados se guardan en un archivo cuyo nombre es una variable, y la extensión que le asignamos es `.data`.

El formato del archivo es el usado por `googleapiclient`, y es relativamente simple. Hay un comentario por línea del archivo, y cada comentario está representado con un formato JSON para incorporar la distinta información asociada al comentario. La Figura 2 muestra una captura de parte del archivo generado³.

```
13 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Pero ecuador paso por lo mismo q n  
14 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Para mi tuvieran q poner mas de ti  
15 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Como seu vizinho, espero que n\u0000  
16 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Massa: ahora viene una etapa nuev  
17 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Schariatti.. Me agradan Mucho tus  
18 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Massa..me parece coherente que pue  
19 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Schiaretti muy contundente en sus  
20 "videoId": "0xQ4vsDK8Hk", "textDisplay": "01:07:00", "textOriginal": "01:07:0  
21 "videoId": "0xQ4vsDK8Hk", "textDisplay": "Conversa de socialista essa frente
```

Figura 2: Captura archivo `.data`

La librería `googleapiclient` extrae distintos metadatos para cada comentario, más allá del texto y el usuario. Analizando cada una de las líneas del archivo `.data`, es decir, uno de los comentarios, podemos ver los campos contenidos en el JSON correspondiente. Las figura 3 y 4 muestra el comentario original y el JSON asociado generado por la librería `googleapiclient`. Aquí dentro podemos encontrar también, si las hubiera, todas las respuestas al comentario original.

²<https://github.com/googleapis/google-api-python-client/tree/main>

³Notar que el archivo completo no respeta el formato JSON, pero sí lo es cada línea del archivo.



Figura 3: Comentario original

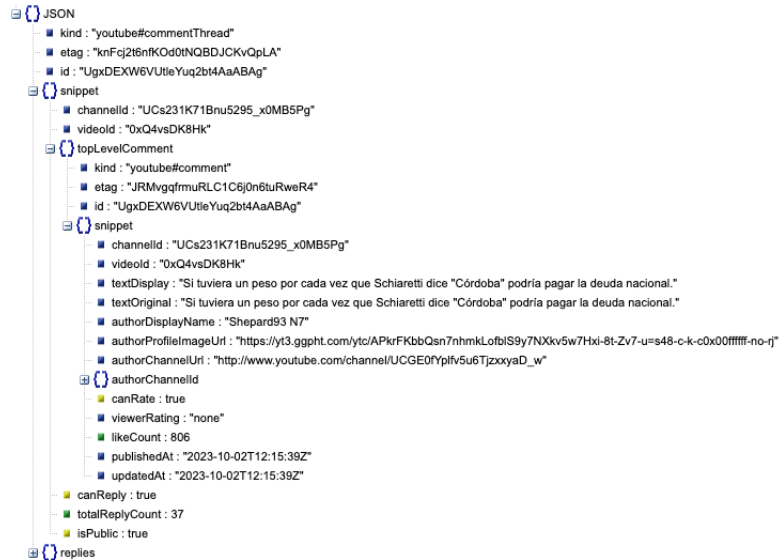


Figura 4: JSON del comentario original

- *Extracción de datos.* Para el armado del *bot* generador de texto vamos a utilizar en principio un solo campo. El dato que nos interesa es el contenido con el texto del comentario, y se puede obtener el string del mismo usando la clave `snippet`. El siguiente paso entonces consiste en hacer un primer procesamiento de la información, separando la relevante para el trabajo de aquella que no será utilizada. El archivo `data_extractor.py` será el encargado de realizar esta tarea. Esencialmente, en base a un archivo de entrada el proceso consiste en extraer el texto de cada comentario y sus respuestas (si las hubiera), guardándolo en un archivo nuevo con un comentario (pero ahora, solo texto) por línea. De esta forma, evitamos tener el resto de la metadata obtenida por la librería. La Figura 5 muestra un ejemplo, donde el resultado es un archivo que solamente contiene los textos de los comentarios, uno por línea.

<pre> 13 "videoId": "0x04vsDK8Hk", "textDisplay": "Pero ecuator paso por lo mismo q n 14 "videoId": "0x04vsDK8Hk", "textDisplay": "Para mi tuvieran q poner mas de ti 15 "videoId": "0x04vsDK8Hk", "textDisplay": "Como seu vizinho, espero que nu00 16 "videoId": "0x04vsDK8Hk", "textDisplay": "Massa; ahora viene una etapa nuev 17 "videoId": "0x04vsDK8Hk", "textDisplay": "Schariatti.. Me agradan Mucho tus 18 "videoId": "0x04vsDK8Hk", "textDisplay": "Massa..me parece coherente que pue 19 "videoId": "0x04vsDK8Hk", "textDisplay": "Schiaretti muy contundente en sus 20 "videoId": "0x04vsDK8Hk", "textDisplay": "01:07:00", "textOriginal": "01:07: 21 "videoId": "0x04vsDK8Hk", "textDisplay": "Conversa de socialista essa frente </pre>	<pre> 13 Pero ecuator paso por lo mismo q nosotros, cambiaron a dolares y estan mucho 14 Para mi tuvieran q poner mas de tiempos a replicas 15 Como seu vizinho, espero que nao elejam seu Bolsonaro, sofremos muito e nao 16 Massa; ahora viene una etapa nueva, "mi gobierno". ** no empezò y ya se desf 17 Schariatti.. Me agradan Mucho tus ideas.. Ojalà hubieras llegado a Balotaje. 18 Massa..me parece coherente que pueda dialogar y unir en vez de deshunir o div 19 Schiaretti muy contundente en sus respuestas. Tiene buenas ideas. Lastimas q 20 01:07:00 21 Conversa de socialista essa frente de trabalhadores... Argentina está assolada </pre>
--	--

Figura 5: Recorte del archivo `.data` (izquierda) y el nuevo archivo `.txt` (derecha)

La función `extract_comments.text` es la encargada de hacer este trabajo. Para ello, recibe dos `strings` con el nombre del archivo `.data` de input y el nombre del archivo `.txt` de output. La función es encargada de abrir ambos archivos, tanto para lectura como escritura de los resultados. Para extraer la información, se recomienda usar la función `loads` del módulo `json`⁴. Dado un `string` que representa un JSON (como los que tenemos en cada línea del archivo `.data`), `loads` toma el `string` como parámetro y como resultado devuelve el objeto JSON correspondiente. La Figura 6 muestra un ejemplo de la utilización.

```
>>> import json
>>> s = '{"a" : 1, "b" : "hola"}'
>>> type(s)
<class 'str'>
>>> j = json.loads(s)
>>> j['a']
1
>>>
>>> j['b']
'hola'
>>>
```

Figura 6: Ejemplo de utilización de `loads`

Es importante notar que la función, en la versión propuesta, permite procesar un único archivo `.data` por vez. En caso de querer usar múltiples archivos, es posible modificar el comportamiento del mismo para considerar múltiples inputs. También es posible utilizar otras herramientas para combinar en un mismo archivo `.data` distintos resultados obtenidos por el downloader, y mantener el archivo actual con el mismo esquema.

3. **Armado e implementación del modelo (5 puntos):** El procesamiento de los datos y el armado del modelo se realiza principalmente en el archivo `graph_generator.py`. El armado del digrafo con el modelo consta de distintas etapas y actividades. En primer lugar, notamos que el archivo cuenta con dos definiciones genéricas con los indicadores de inicio y fin luego de los imports que pueden ser accedidos desde cualquier función sin necesidad de pasarlos como parámetros.

Supongamos que el archivo `.txt` es el siguiente, donde ahora tenemos dos comentarios (uno por línea):

```
He walks down eerie roads.
He hears scary music.
```

Las etapas y funciones son las siguientes:

- *Procesamiento y tokenización de los comentarios.* El primer paso es transformar esos strings en los denominados *tokens*, que en este contexto funciona como sinónimo de *palabra*. Serán las unidades que definirán los vértices del grafo. La función `process_comments` toma como único parámetro el nombre del archivo `.txt` con los comentarios extraídos en la etapa previa. Las tareas a realizar por la función son las siguientes:
 - a) Abrir el archivo y recorrer todas las líneas para procesarlas.

⁴<https://docs.python.org/3/library/json.html>

- b) Para cada línea, agregar al principio y al final los indicadores de inicio y fin (incluyendo los espacios que correspondan). En el ejemplo, la primera línea debería convertirse en el siguiente **string**:

```
__beg__ __beg__ He walks down eerie roads. __end__ __end__
```

Para esto se pueden usar variables definidas de forma genérica.

- c) El siguiente paso es tokenizar el texto de la línea actual. Debido a la complejidad que tienen la escritura de los comentarios, vamos a usar una función de una librería muy utilizada para procesamiento del lenguaje, **nltk**. usando la función **word_tokenize**⁵, a partir de un **string** nos devuelve una lista de **string**, donde cada uno de ellos es un *token*. Mostramos a continuación un ejemplo de su utilización:

```
from nltk.tokenize import word_tokenize
s0 = '__beg__ He walks down eerie roads. __end__'
tokens = word_tokenize(s0)
# resultado en tokens:
['__beg__', 'He', 'walks', 'down', 'eerie', 'roads', '.', '.__end__']
```

- d) En base a esta lista de tokens, se deben generar los vértices del digrafo inducidos por el comentario correspondiente, para luego poder agregar los ejes al mismo. Para ello, de la lista de tokens, vamos a generar la lista de vértices (que, además, capturan el orden de aparición en el texto) de la siguiente forma:

```
[('__beg__', '__beg__'), ('__beg__', 'He'), ('He', 'walks'),
 ('walks', 'down'), ('down', 'eerie'), ('eerie', 'roads'),
 ('roads', '.'), ('.', '.__end__'), ('.__end__', '.__end__')]
```

Esta versión corresponde a la implementación de vértices con tuplas; una análoga se obtendría en caso de considerar otra opción.

- e) Finalmente, agregamos la lista de vértices a la lista de resultados.
- f) La función debe retornar un **list** de **list** de **tuplas** (o **strings**), donde la posición *i* de la lista principal corresponde a la lista que contiene los vértices de la fila *i* del archivo de entrada. Siguiendo con el ejemplo, el resultado debería ser el siguiente.

```
[(['__beg__', '__beg__'), ('__beg__', 'He'), ('He', 'walks'),
 ('walks', 'down'), ('down', 'eerie'), ('eerie', 'roads'),
 ('roads', '.'), ('.', '.__end__'), ('.__end__', '.__end__')],
 [('__beg__', '__beg__'), ('__beg__', 'He'), ('He', 'hears'),
 ('hears', 'scary'), ('scary', 'music'), ('music', '.'),
 ('.', '.__end__'), ('.__end__', '.__end__')]]
```

- *Armado inicial del digrafo.* El armado del modelo lo separamos en dos partes. La primera parte consta de las siguientes dos funciones:

- a) **generate_graph(tknzed_text)**. El parámetro es la lista de lista de vértices generada en el paso anterior. La función debe retornar el digrafo con los ejes de cada bigrama presente en todo el input. Cada eje tendrá un peso asociado, que será la cantidad de veces que se repite el digrama. Esta función simplemente recorre una a una las listas de tokens recibidas como parámetro, y para cada una llama a una función auxiliar encargada de agregar la lista de tokens (que representa uno de los comentarios) al digrafo.

⁵<https://www.nltk.org/api/nltk.tokenize.html>

- b) `add_text_to_digraph(text,D)`. Recibe como parámetro un comentario tokenizado en `text` y lo agrega al digrafo `D`. Notar que al modificar `D` internamente en la función, los cambios tienen efecto fuera y no es necesario retornar un nuevo grafo. Esta función debe no solo agregar el eje, sino también ir ajustando los pesos para que reflejen la cantidad de apariciones del correspondiente bigrama representado por el eje. Esta función debe ser llamada por `generate_graph`.

Siguiendo con el ejemplo, luego de llamar a la línea `generate_graph(tknzd_text)` en la función `main`, el digrafo contiene todos los vértices, los ejes y los respectivos pesos w_{ij} representan la cantidad de veces que ese eje aparece en toda la base usada como input. Notar que en el ejemplo, por ser pequeño, este valor toma siempre valor 1 pero no es el caso general. Por ejemplo: si *He walks down* apareciera en dos comentarios distintos, entonces el peso del arco `((He, walks), (walks,down))` sería 2.

- *Cálculo de probabilidades de transiciones*. Para completar el armado de la Cadena de Markov, solamente convertir la cantidad de apariciones de cada eje en frecuencias en cada caso. Para ello, se pide completar la función `adjust_out_edges`, que toma como parámetro un vértice del digrafo y ajusta las probabilidades de los arcos salientes del mismo. Notar que las probabilidades de un arco $(i,j) \in A$ pueden calcularse simplemente como la proporción entre el peso actual w_{ij} y la suma de todos los arcos salientes del vértice i . Para revisar la implementación, notar que la suma de los nuevos pesos de los arcos salientes del vértice recibido como parámetro debe ser 1.0.

La función `adjust_out_edges` es una función auxiliar de `calculate_markov_chain`, que solamente toma como parámetro el digrafo `D` y aplica esta función a cada vértice. Notar que, al finalizar, al considerar el ejemplo el grafo resultante debería ser el de la figura mostrado junto con la explicación del modelo.

Una vez finalizado el proceso, el modelo resultante es exportado usando el formato `.gml`.

4. **Experimentación y análisis (2 puntos)**: con el modelo construido, solo nos queda usarlo para simular comentarios y analizar su comportamiento. Para la primera tarea, contamos con el archivo `bot.py`. Este archivo lee el modelo exportado en formato `.gml` y simula un camino aleatorio usando la función `simulate_comment`. Se provee una función auxiliar, `get_next_word` que, dado el digrafo con el modelo y una palabra que corresponda a un nodo del mismo, elige aleatoriamente cuál será la próxima palabra siguiendo las probabilidades calculadas. La palabra recibida como parámetro no puede ser `(__end__, __end__)`, ya que el mismo indica el fin del comentario y no posee sucesores.

En base a esta función, `simulate_comment` empieza el comentario con el indicador de inicio `(__beg__, __beg__)` y debe generar la siguiente palabra de manera aleatoria hasta llegar a que la palabra actual sea el indicador de fin, `(__end__, __end__)`. Notar que pueden aplicarse algunas reglas simples para lograr comentarios más realistas, como manejar con cuidado los espacios en caso de tener símbolos de puntuación o fin de oración.

Una vez completada esta función, se pide que el grupo experimente con los datos elegidos generando gran cantidad de comentarios. Sobre los mismos, se pide aplicar un análisis cualitativo identificando casos donde (i) el texto generado sea razonable y pueda pasar

como verdadero, (ii) casos donde los comentarios generados no pasen un primer filtro de calidad, (iii) un análisis general del tipo de comentarios generados por el/los video/s en función de los resultados, (iv) análisis de los casos en los que falla y posibles razones que justifiquen el comportamiento. Cualquier otro análisis adicional que el grupo desee realizar puede ser incorporado, junto con estos, al informe del trabajo.

El informe deberá contener, además, una descripción sobre las decisiones tomadas para la implementación de las funciones que consideren más complicadas del trabajo y una introducción contando la idea detrás del TP y de la elección del (o los) video(s) utilizados como input.

Modalidad de entrega

Además del código con las implementaciones es posible entregar un mini informe de no más de tres carillas en donde detallen, en caso de considerarlo necesario, decisiones que tomaron para resolver uno o más ejercicios y que ayuden a entender la estrategia de resolución.

Fecha de entrega

miércoles 26 de junio de 2024, hasta las 23:59 hs.. Subir el trabajo (informe + código) al campus (uno por equipo). Se puede subir tanto los archivos por separado como juntos en un .zip.

Importante: El horario es estricto. Los trabajos recibidos después de la hora indicada serán considerados re-entrega.